

# Punteros

Por: Ricardo Cuellar



# ¿Qué son punteros en Go?

Un puntero es un valor de tipo `*T` que guarda la dirección donde vive un `T`.

Permite trabajar con el mismo dato, no con una copia.

**0x0007FF8E3C3821**

*Un puntero guarda una dirección, no el valor*



# Los 3 símbolos de la legión puntero

Hay tres símbolos que debes aprender para poder usar correctamente los punteros:

**&x** : Obtiene la dirección de x

**\*p** : Entra a la dirección y usa el valor real

**nil** : No apunta a nada

**&**

**\***

**nil**



# En Go todo se pasa por valor

La diferencia es que pasas:

- $T \rightarrow$  Copia del dato
- $*T \rightarrow$  Copia de la dirección (apunta al original)



# Bits y Bytes

**bit:** Unidad mínima de información. Solo puede ser 0 o 1 (cero o uno).

**Byte:** Un grupo de 8 bits. Es la unidad básica con la que normalmente medimos memoria/archivos.

**1 byte = 8bits**

**2 bytes = 16bits**

...

**1 KB = 1,024 bytes**

**1 MB = 1,024 KB**

0

1

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|---|---|

x | y



# Nota mental

**&x = “dame la dirección de memoria donde está x”**

**\*p = “ve a esa dirección y lee/escribe el valor”**



# Como se asigna una variable

```
var x int 32 = 10  
var y bool = true
```

Bytes

Valor  
Dirección

|   |   |   |    |   |
|---|---|---|----|---|
| 0 | 0 | 0 | 10 | 1 |
| 1 | 2 | 3 | 4  | 5 |

x

| y



# Ejemplo con struct



methods.go

```
type User struct{ Name string }
```

```
func RenameCopy(u User, name string) { u.Name = name }
```

```
func RenamePtr(u *User, name string) { u.Name = name }
```

```
u := User{Name:"Ana"}
```

```
RenameCopy(u, "Beto")    // no cambia afuera
```

```
RenamePtr(&u, "Beto")    // sí cambia afuera
```



# Ejemplo con métodos: receiver



methods.go

```
func (u *User) SetName(name string) { u.Name = name }
```



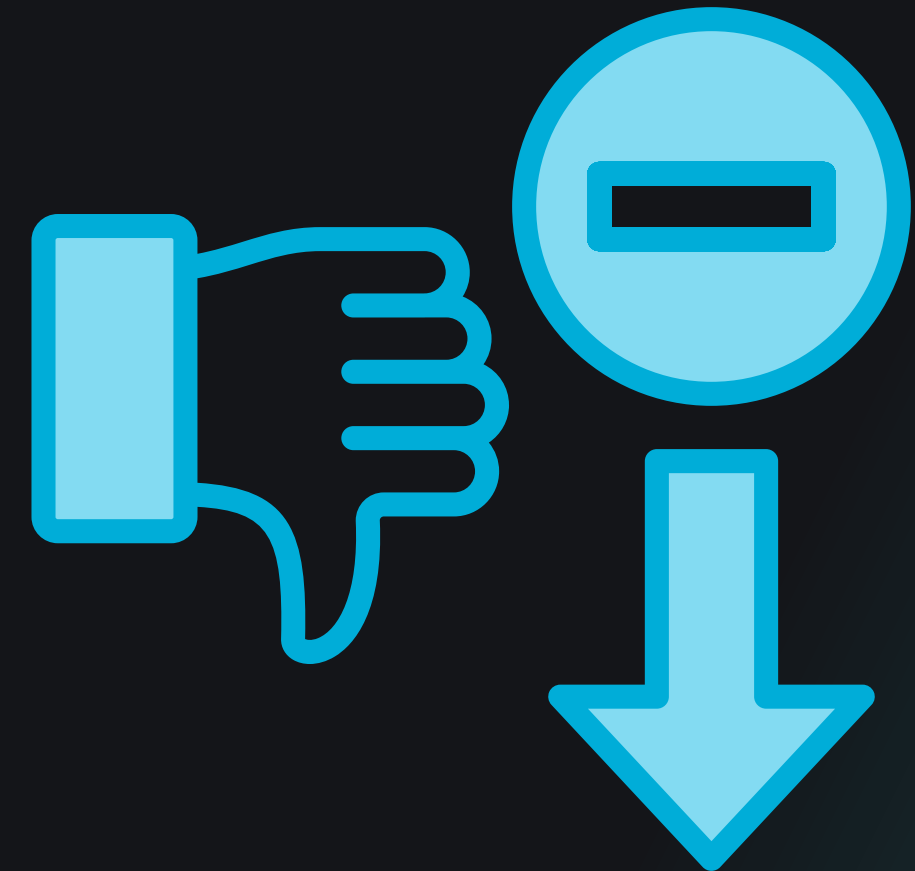
# Ventajas

- Mutar un valor desde funciones o métodos.
- Evitar copias grandes o structs muy pesados.
- Representar ausencia con nil



# Desventajas

- Más complejidad mental.
- nil puede causar panic si no lo diferenciamos bien.
- No siempre se mejora el rendimiento. Te recomiendo priorizar la claridad.



# Errores comunes

- Diferenciar nil
  - u.Name cuando u == nil
- Usar punteros para verme pro
  - Si no los necesitas no los uses.
- Confundir slices/maps:
  - Se sienten como referencia pero reasignar dentro de la función no cambia el original si no se devuelve.



# ¿Cuándo usarlos?

- Se necesita mutar.
- Necesitas nil.
- El costo de copiar es significativo.
- Cuando tengas valores opcionales.
- Cuando API estándar decide por ti.



# Gracias



[www.devtalles.com](http://www.devtalles.com)



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

